

Bookshop POS

Standing rules (apply to every phase)

1. Do NOT change `db/schema.ts` without asking me first and explaining why. The audit-log pattern (`stock_movements`) and frozen sale prices (`sale_items.unitPrice`) are intentional design decisions, not oversights.
 2. Stop and ask if a required env var, credential, or config value is missing. Never invent placeholder secrets and continue silently.
 3. After finishing a phase, summarize what was built, list any new env vars or dependencies added, and explicitly tell me what to test manually before we move to the next phase. Then STOP — do not start the next phase unprompted.
 4. Keep server-side validation (Zod) on every route that writes to the DB. Never trust client-sent prices, quantities, or IDs for the sell flow — always recompute totals server-side from the DB.
 5. All stock quantity changes must go through a DB transaction and must insert a corresponding `stock_movements` row. Never update `products.quantity` directly outside a transaction.
-

Phase 0 — Setup (Day 1)

1. **Init the project**

bash

```
npx create-next-app@latest bookshop-pos --typescript --app --tailwind
cd bookshop-pos
```

2. **Get a Postgres instance** — sign up for **Neon** or **Supabase** (free tier, instant, no local install needed). Grab the connection string.
3. **Install core deps**

bash

```
npm install drizzle-orm postgres
npm install -D drizzle-kit
npm install zod
npm install next-auth@beta
```

4. **Init git**, push to GitHub early.

Phase 1 — Database schema (Day 1–2)

Define tables in Drizzle: `products`, `sales`, `sale_items`, `stock_movements`, `users`. Run migrations. Seed a handful of fake books so you have data to build against immediately — don't build UI against an empty DB.

Phase 2 — Auth

Set up Auth.js (NextAuth) with a credentials provider against the `users` table already in `db/schema.ts`.

- Login page at `/login` (email + password)
- Passwords hashed with `bcrypt`; update `db/seed.ts`'s placeholder hash with a real one and tell me the seeded admin login credentials
- Two roles from the `users.role` enum: `admin`, `cashier`
- Middleware that redirects unauthenticated users to `/login`, and restricts `/admin/*` routes to `role = admin` only
- Cashiers land on the sell screen after login; admins land on a dashboard
- Session should include `id` and `role` so route handlers can check permissions without an extra DB query every time

Stop after this phase. I will log in as both roles before continuing.

Phase 3 — Stock management CRUD

Build the admin-only product management screens.

- `/admin/products` — table view: title, author, quantity, price, low-stock indicator (highlight rows where `quantity <= reorderThreshold`)
- Search/filter by title, author, ISBN, category
- Add product form and edit product form, validated with `Zod`
- "Adjust stock" action (separate from editing the product record) that:
 - Takes a quantity delta and a reason (`restock`, `adjustment`, `return`)
 - Runs in a transaction: updates `products.quantity` AND inserts a `stock_movements` row — never one without the other
- Do NOT add a delete-product button yet — deactivate/archive only (add an `isActive` boolean if needed) so sales history stays intact. Ask me before changing the schema for this.

Stop after this phase. I will add/edit a few products manually before continuing.

Phase 4 — Sell screen (core feature — take the most care here)

Build the cashier-facing checkout flow at `/sell` (or the root route for the `cashier` role).

- Global keydown listener that captures fast barcode-scanner input (digits + Enter), not a manually-focused input per field
- Manual search fallback for books without a scannable barcode/ISBN
- Cart state on the client; running total updates live as items are added
- "Undo last item" and "clear cart" actions
- Checkout endpoint (`POST /api/sales`) that:
 1. Validates the cart server-side (real product IDs, real current prices — never trust client-sent prices)
 2. Opens ONE DB transaction that: locks each product row, checks `quantity >= requested`, rejects the whole sale if any item is insufficient, decrements stock, inserts the `sales` row, inserts `sale_items` rows (with price frozen at time of sale), inserts `stock_movements` rows with `reason = 'sale'` and `relatedSaleId` set
 3. Returns a clear error if stock was insufficient, naming which item
- Show a confirmation/receipt view after successful checkout (on-screen is enough for now — printing comes later)
- Handle the double-submit case (cashier double-scans checkout) — disable the button during the request, and make the endpoint safe to reject a duplicate rapid submission

This is the most important phase. After building it, write and show me a short test plan (e.g. "sell 2 units of a book with only 1 in stock and confirm it's rejected and stock is unchanged") rather than just saying it's done.

Stop after this phase. I will manually test several sell scenarios, including an insufficient-stock case, before continuing.

Phase 5 — Analytics dashboard

Build `/admin/analytics` (or fold into the main admin dashboard).

- Sales totals: today, this week, this month — query `sales/sale_items` grouped by date, don't recompute from `products`
- Top-selling products: `SUM(sale_items.quantity) GROUP BY product, ordered descending`, for a selectable date range (day/week/month)
- Low-stock list: products where `quantity <= reorderThreshold`

- "Nearing stockout" estimate: use recent `stock_movements` (`reason='sale'`) to estimate a sell-rate per product, then estimate days-until-zero from current quantity — show this as a soft estimate, not a guarantee
- Simple charts are fine (a lightweight library, or plain tables/bars) — don't over-invest in visuals yet

Stop after this phase for review.

Phase 6 — Low-stock notifications

- v1 (do this first): a persistent banner/badge on the admin dashboard computed on page load from the same low-stock query as Phase 5 — no new infrastructure needed
- v2 (only if I ask for it): a Vercel Cron job hitting a `/api/cron/check-stock` route daily, sending an email (or similar) if any product crosses its reorder threshold. Ask me which notification channel I actually want before building this — don't assume email.

Stop after v1 for review before touching v2.

Phase 7 — Polish & hardening

- Zod validation audit: confirm every write route validates input, especially the sell endpoint
- Error and empty states on every list/table
- Confirm the checkout transaction handles a mid-request crash or network drop without leaving stock in a bad state (test this explicitly)
- Basic printable receipt (print-friendly HTML/CSS view is enough — do not build thermal-printer/WebUSB integration unless I explicitly ask, that's a separate hardware-dependent effort)

Stop after this phase for review.

Phase 8 — Deploy

- Confirm production `DATABASE_URL` and `AUTH_SECRET` (or equivalent) are set in Vercel's environment variables, not committed to the repo
- Deploy to Vercel
- Walk me through a full smoke test in production: log in as cashier, sell an item, confirm stock decremented, log in as admin, confirm the sale shows in analytics
- Only after I confirm the smoke test passes, treat the deploy as done

Start on Phase 2 now. Stop after it's done and summarized, per the standing rules above.